Strategy Addendum to HANDOFF_ROADMAP_v4

User: Joey Clark | **Written:** 2026-07-19 **Purpose:** Extends the v4 roadmap with four sourcing strategies. The executing agent (Claude Code) should read this alongside the roadmap. Sections 2 and 3 contain build tasks; sections 1 and 4 are operating strategy for Joey with a small supporting build task each.

---

1. LinkedIn strategy: optimized manual links

Principle: LinkedIn cannot be scraped reliably or within its terms of service. The winning architecture is generated URLs plus a short daily human pass, not automation theater. The pipeline's job is to make the manual pass take 5 minutes and show only what is new.

1.1 URL anatomy (the building blocks)

Base: `https://www.linkedin.com/jobs/search/?` plus parameters joined with `&`:

Parameter	Meaning	Values Joey uses
`keywords=`	Search string, supports quotes and OR	e.g. `keywords=%22chief%20of%20staff%22`
`f_TPR=`	Time posted, in SECONDS	`r86400` = 24h, `r259200` = 3 days, `r604800` = week
`f_WT=`	Workplace type	`2` = remote, `3` = hybrid, `1` = onsite
`f_E=`	Experience level	`4` = mid-senior, `5` = director, `6` = executive. Use `f_E=4,5,6`
`geoId=`	Location	Argentina `100446943`, Mexico `103323778`, Latin America region `104514572`, United States `103644278`. Verify by running a manual search and copying the geoId from the resulting URL

`sortBy=DD`	Sort newest first	Always include
`f_JT=F`	Full-time only	Optional

Boolean keywords work: `("chief of staff" OR "director of strategy") NOT intern` URL-encoded in the keywords param.

1.2 The daily 24-hour matrix

Build task for Claude Code: upgrade `linkedin_links.py` to emit a single markdown file, `LinkedIn_Daily_Links.md`, containing one past-24h link (`f_TPR=r86400&sortBy=DD`) per cell of this matrix:

- Rows (role clusters, one boolean string each): (1) Chief of Staff / Strategic Initiatives, (2) Country Manager / General Manager / Market Entry, (3) Partnerships / Business Development (director+ terms only), (4) Strategy and Operations / Corporate Development, (5) VC Platform / Operating Partner / Portfolio Operations.
- Columns (geographies): Argentina, Mexico, Latin America region with `f_WT=2`, and Remote-worldwide (keywords include “LATAM” OR “Latin America” with `f_WT=2`).

That is roughly 20 links. Also emit a weekly version of the same matrix with `r604800` as a catch-all backstop.

1.3 Operating strategy for the daily pass (Joey’s routine, 5-10 min)

1. Open the daily links file, click through the 20 links. Because each shows only the last 24 hours sorted newest-first, most return zero to five results. Total review load is typically under 30 postings a day.
2. Anything interesting: use LinkedIn’s Save button. Saved jobs become the queue.
3. For any saved job, check the “connections at company” module before applying. A second-degree path to the hiring manager or a platform/talent person converts far better than a cold application. Warm route before cold, per standing principle.
4. Set LinkedIn’s native job alerts ON for the two or three highest-yield searches only (alerts beyond that become noise). The generated links replace broad alerts.
5. Speed rule: for a Tier 1 role (CDMX or BA based) posted within 24 hours, same-day application beats a perfect application on day 4. Tailoring depth scales with fit score, not with available time.

1.4 What NOT to do

No scraping, no browser automation against LinkedIn, no third-party "LinkedIn API" resellers of profile data. Detection risk to the account is not worth it and the account itself (network, recruiter visibility, saved searches) is an asset the search depends on. Google Jobs via SerpAPI (section 4) is the legitimate programmatic complement because LinkedIn postings frequently syndicate into Google Jobs results.

---

2. Week-over-week diff, upgraded to daily-new detection (build task)

Why: applications inside the first 24-72 hours of a posting get read at materially higher rates. The weekly report bundles a Monday posting into Sunday delivery, wasting the freshness advantage on exactly the corpus companies where fit is already validated.

Spec for Claude Code (add to Phase 1 as item 5):

1. `diff.py` : after any `discover.py` run, compare current API results against the previous run's snapshot (`snapshots/api_results_{date}.csv`). Output `new_since_last_run.csv` with only postings whose job ID or URL was not present before.
2. Daily fresh sweep: the ATS API sweep costs 2-3 minutes and no model tokens. Add a launchd job (or a one-line manual command: `python3 JC3/run.py --user joey --variant latam --fresh-only`) that runs discover + diff + health daily, and if `new_since_last_run.csv` is non-empty, writes a short `FRESH_{date}.md` alert file listing company, title, location, URL. No scoring on the daily run; Joey eyeballs it in one minute.
3. The full weekly run (fetch, score, package, Master update) stays weekly. The daily sweep is purely an early-warning layer on corpus companies.
4. Metrics addition: track median days-from-posting-to-Joey-seeing-it. Target under 2 days for corpus companies.

---

3. VC portfolio job boards as discovery sources (build task, Phase 3 priority 1)

Most of the target funds run aggregated portfolio job boards on Getro or Consider. These are structured, fetch-friendly with plain requests or light Playwright, pre-filtered to exactly the right companies, and often carry roles not yet indexed anywhere else. Add `portfolio_boards.py` as a discovery source.

Boards to integrate (resolve exact URLs at build time; typical pattern is jobs.{fund}.com or {fund}.getro.com; do not trust remembered URLs):

LATAM core: Kaszek, monashees, NXTP, ALLVP, Valor Capital Group, Atlantico, Nazca, QED Investors, SoftBank Latin America, Endeavor (jobs board spans markets, filter to Mexico/Argentina). US-LATAM bridge and adjacent: a16z, General Catalyst, and Founders Fund portfolio boards, filtered to remote and LATAM locations only, given the dual-use/defense sector priority.

Integration rules:

1. Getro and Consider boards usually expose a JSON endpoint behind the page. Have Claude Code inspect network traffic once per board and prefer the JSON endpoint over HTML parsing.
2. Filter at ingest by role-family keywords and location tiers, same gates as the ATS sweep.
3. Every company that appears with a relevant role gets appended to companies.json via corpus_append.py with `latam_relevance` set appropriately and its ATS slug resolved by verify_ats.py. The boards thus serve double duty: direct job source and corpus growth engine.
4. Provenance column value: `portfolio_board:{fund}`. Track yield per board in metrics; drop boards that produce nothing for 6 weeks.

Also add in the same script, as simple fetch targets: Getonbrd API (per roadmap Phase 3) and Hireline.io for Mexico.

---

4. SerpAPI Google Jobs: step-by-step for Joey (non-technical)

What it is: SerpAPI is a paid service that runs Google searches for you and returns clean, structured results, legally and without scraping headaches. Its Google Jobs endpoint returns the same aggregated postings a person sees in Google's jobs box, which includes many LinkedIn-syndicated and direct-career-page postings the ATS sweep misses. This is the legitimate programmatic complement to the manual LinkedIn pass.

Cost reality: the free plan currently includes a small monthly allowance (on the order of 100-250 searches; confirm the current number at signup). Your volume is roughly 15-25 queries per weekly run, about 60-100 per month, so the free plan likely covers it. If not, the entry paid plan is about $25/month for 1,000 searches. Either way, far below a Max subscription.

Steps (one-time setup, 15 minutes):

1. Go to serpapi.com and create an account (email + password). No credit card needed for the free plan.
2. After signing in, the dashboard shows "Your Private API Key," a long string of letters and numbers. Copy it.
3. On your Mac, open Claude Code in the repo and say: "Create a file called .env in the repo root containing SERPAPI_KEY=" and paste the key. Then: "Make sure .env is listed in .gitignore." (This keeps the key out of GitHub. Never paste the key into any file that gets committed.)
4. Tell Claude Code: "Build serpapi_jobs.py as a discovery source per section 4 of STRATEGY_ADDENDUM_v4.md."
5. That is your entire involvement. From then on the weekly run includes it automatically.

Spec for Claude Code (what step 4 builds):

- Endpoint: `https://serpapi.com/search` with `engine=google_jobs`, `api_key` from .env, `q` = role phrase, `location` = "Mexico City, Mexico" / "Buenos Aires, Argentina", plus a remote-LATAM query set using q strings like `"chief of staff" remote LATAM`. Use the `date_posted` filter parameter for past-week freshness (check current parameter name in SerpAPI's Google Jobs docs at build time).
- Query budget: cap at 25 queries per weekly run (5 role clusters x 5 geo/remote variants). Log usage.
- Each result includes title, company, location, description snippet, and apply links. Treat these as DISCOVERY rows: they still pass through health.py and fetch_jds.py like any search-sourced row (SerpAPI results can be stale; the pipeline's precision gates apply).
- New companies surfacing here feed corpus_append.py, same as every other source.
- Provenance value: `serpapi_google_jobs`. Track yield; if it underperforms portfolio boards and Getonbrd after 4 weeks, cut the query budget rather than the source.

Priority order if building incrementally

1. Week-over-week / daily-new diff (section 2): highest impact per hour of build.
2. LinkedIn daily links upgrade (section 1.2): trivial build, immediate daily value.
3. Portfolio boards + Getonbrd (section 3): the recall engine.
4. SerpAPI (section 4): additive breadth once 1-3 run clean.

End of addendum.